



INSTITUTO SUPERIOR DE ENGENHARIA DE LISBOA (ISEL)

DEPARTAMENTO DE ENGENHARIA INFORMÁTICA (DEI)

MEIM

MESTRADO EM ENGENHARIA INFORMÁTICA E MULTIMÉDIA
REPRESENTAÇÃO E PROCESSAMENTO DO CONHECIMENTO

Trabalho Prático Final

Grupo 01

Miguel Alcobia (50746)

Fábio Pestana (50756)

Docente

Professor Paulo Trigo Silva

Junho, 2026

Índice

Índice	i
Lista de Figuras	ii
1 Introdução	1
2 Fase 1 - Construir TBox	2
3 Fase 2 - Criação da ABox	4
4 Fase 3 - Interrogações, Inferências e Página <i>Web</i>	8
4.0.1 Inferências com DL	8
4.0.2 Interrogações SPARQL	11
4.0.3 Página <i>Web</i>	16
5 Conclusão	18

Lista de Figuras

2.1	Teste com DL Query para ver os Autores	3
4.1	DL Query para ver obras de um ano específico	9
4.2	DL Query para ver obras de um autor específico	9
4.3	DL Query para ver autores existentes	10
4.4	DL Query para ver que autores têm nomes	10
4.5	DL Query para ver obras de uma editora específica	11
4.6	Query SPARQL para ver o nome de cada Person	12
4.7	Resultado da Query SPARQL para ver o nome de cada Person	12
4.8	Query SPARQL para ver cada Publication, o seu título e ano da publicação	13
4.9	Resultado da Query SPARQL para ver cada Publication, o seu título e ano da publicação	13
4.10	Query SPARQL para ver quantos livros se publicou em cada ano	14
4.11	Resultado da Query SPARQL para ver quantos livros se pu- blicou em cada ano	14
4.12	Query SPARQL para ver quantos documentos há de cada tipo	15
4.13	Resultado da Query SPARQL para ver quantos documentos há de cada tipo	15
4.14	Página principal da aplicação <i>web</i>	17
4.15	Página com as publicações de Edsger W. Dijkstra.	17

Capítulo 1

Introdução

Este trabalho foi realizado no âmbito da Unidade Curricular de Representação e Processamento do Conhecimento tem como objetivo o desenvolvimento de um repositório de referências bibliográficas pessoais, como o DBLP e organizando-os de acordo com uma ontologia adaptada a partir do laboratório SWAT.

Na primeira fase do trabalho, construiu-se uma TBox que define os conceitos fundamentais da ontologia, as propriedades e as restrições relevantes.

Foram realizados pequenos ajustes para estabelecer uma correspondência entre a nomenclatura dos *schemas* da ontologia criada e a utilizada pelo DBLP, garantindo assim compatibilidade e coerência semântica. A construção e validação da TBox foram apoiadas pelo uso do Protégé, conforme explorado nas aulas práticas 8 e 9.

Na segunda fase, construiu-se a ABox através da extração de dados reais dos SPARQL-*endpoints* do DBLP. Este processo envolveu a criação de um grafo local, que depois foi populado com os resultados das *queries* SPARQL. Esta fase foi realizada com base nos conhecimentos adquiridos na aula prática 7.

Na terceira e última fase, a ontologia resultante (TBox e ABox) foi disponibilizada como um SPARQL-*endpoint*, permitindo a realização de interrogações e inferências — como a classificação automática de Publication, Unpublished, Autor e Autor_Org. Esta fase foi concretizada através da exportação com o RDF4J e da criação de uma página HTML baseado no servidor CherryPy, seguindo o que foi feito nas aulas práticas 5, 6 e 7.

Capítulo 2

Fase 1 - Construir TBox

Na primeira fase do trabalho, foi pedido para criar uma TBox para o contexto do problema. A ontologia foi baseada na já existente `dblp.owl.xml`.

Acrescentaram-se alguns conceitos primitivos, como `Document`, `Organization` e `Person`.

Podemos ter documentos publicados ou não publicados; portanto, criaram-se duas subclasses de `Document`: `Publication` e `Unpublished`.

Em seguida, definiram-se os tipos de `Publication` existentes: `Article_in_Journal` (Artigos em periódicos/revistas), `Article_in_Proceedings` (Artigos em Atas) e `Book` (Livro). Todas elas são disjuntivas entre si, pois uma publicação só pode ter um destes tipos exclusivamente.

Em termos de propriedades, seguiram-se as indicações do enunciado de manter algumas das já existentes e ter atenção, às resultantes das respostas das *queries* SPARQL. Contudo, como os *links* dos prefixos das *queries* SPARQL estavam indisponíveis, algumas propriedades foram deixadas de lado: `has-web-address`, `author_full_name` (não se encontrou alternativa) e `has-volume`.

Vale ressaltar que, em vez da ontologia AKT (Aktors), o link disponível em <http://dblp.rkbexplorer.com> estava indisponível. Usou-se o link <https://sparql.dblp.org/> em alternativa e, por esse motivo, existem algumas inconsistências e mudanças em relação ao que o enunciado pedia, mas foram feitos os ajustes necessários.

Para `has-web-address` e `has-volume` descobriu-se durante a escrita deste relatório que tinham um equivalente: `ee` e `volume` respetivamente. Contudo, dado o tempo limitado, optou-se por deixar estas duas propriedades de lado,

visto que não afetavam o objetivo principal do trabalho.

Depois acrescentou-se o conceito de **Autor**, que se definiu com Lógica de descrição, que equivale a uma Pessoa que seja autora de algum Documento (`Person and (author_inv some Document)`). Já **Autor_Org** foi definido como qualquer Autor que tenha registo da sua afiliação (tem uma relação `affiliation`) (`Autor and (affiliation some Organization)`).

Definiram-se os conceitos **Publication** e **Unpublished**: `Document and (publisher some Organization)` e `Document and (publisher value none)` respetivamente. Ou seja, uma Publicação é um Documento publicado por uma Organização e um não publicado significa que não tem editora.

Por fim, fizeram-se os testes solicitados no enunciado.

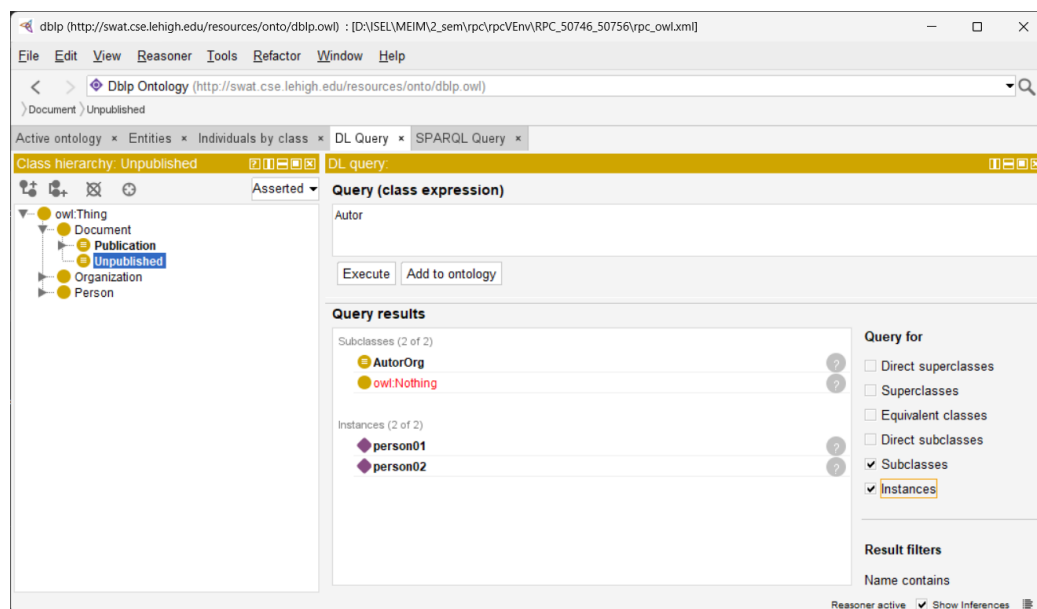


Figura 2.1: Teste com DL Query para ver os Autores

Capítulo 3

Fase 2 - Criação da ABox

A segunda fase do projeto consistiu na construção de uma ABox para a ontologia desenvolvida na Fase 1. O objetivo principal foi povoar a ontologia com dados reais obtidos a partir de fontes externas, utilizando o SPARQL *endpoint* do repositório DBLP.

O CiteeSeek foi descontinuado e não se encontrou alternativa.

A solução para esta fase implementada no *script* `GG_fase2_grafo.py` e está dividido em três etapas:

- **Criação do grafo de *input*** – contém os autores (pessoas) que servirão de ponto de partida para a consulta ao DBLP.
- **Consulta ao *endpoint* SPARQL do DBLP** – A consulta é feita para cada autor para se obter a lista das suas publicações.
- **Construção do grafo de *output*** – mapeia os resultados obtidos para a ontologia RPC, gerando *individuals* e propriedades.

O código desenvolvido teve por base o realizado na aula prática 7, que já servia para fundamentar o trabalho final.

De forma geral, o código tem uma classe (`DBLPClient`) e funções auxiliares. Entre elas:

- `normalizar_para_uri(nome)` - Normaliza *strings* para serem usados em URIs. Converte os nomes para minúsculas, remove acentos e substitui caracteres especiais por *underscores*.

- `gerar_uri_pessoa(nome_autor)` - Gera uma URI no *namespace* RPC para uma pessoa a partir do seu nome.
- `gerar_uri_publicacao(titulo, ano=None)` - Gera uma URI no *namespace* RPC para uma publicação baseada no título e ano opcional.
- `gerar_uri_organizacao(nome_organizacao)` - Gera uma URI no *namespace* RPC para uma organização a partir do seu nome.
- `processar_multiplas_publishers(editora_str)` - Processa *strings* que podem conter múltiplas editoras separadas por vírgulas e retorna uma lista com os vários nomes.
- `mapear_classe_dblp_para_rpc(tipo_dblp)` - Converte o tipo de publicação do DBLP para a classe correspondente na ontologia RPC.
- `declarar_propriedades_se_necessario(g, propriedade, tipo)` - Declara propriedades no grafo como `ObjectProperty` ou `DatatypeProperty` se ainda não existirem.
- `criar_grafo_input()` - Cria o grafo RDF de entrada com as instâncias da classe `Person` para cada autor.
- `criar_grafo_output_com_dblp()` - Constrói o grafo RDF de saída e mapeia as publicações do DBLP para a ontologia RPC.

A classe `DBLPClient` tem, por sua vez, os seguintes métodos:

- `consultar(query)` - Executa uma consulta SPARQL no *endpoint* do DBLP e retorna os resultados em formato JSON.
- `_processar_resultado(resultado)` - Processa os resultados da *query*, extraindo variáveis e *bindings*.
- `procurar_autores()` - Consulta o DBLP para obter uma lista de autores, com limite configurável.

- `procurar_publicacoes_por_autor(nome_autor)` - Consulta todas as publicações de um autor específico, incluindo título, ano, tipo, editora e journal.

As alterações feitas devem-se ao mapeamento feito. Como referido anteriormente, este mapeamento deve-se ao facto de que a ontologia do DBLP tem nomes diferentes da criada na Fase 1. Portanto, descarregou-se o *schema* do DBLP (que está disponível na pasta do projeto em `schema.rdf` e procurou-se pelas melhores correspondências.

As tabelas abaixo mostram as correspondências realizadas:

Tabela 3.1: Mapeamento de Classes DBLP para RPC

Classe DBLP	Classe RPC
Article	Article_in_Journal
Inproceedings	Article_in_Proceedings
Incollection	Article_in_Proceedings
Book	Book
Editorship	Document
Reference	Document
Data	Document
Informal	Document
(<i>outros ou indefinido</i>)	Document

Tabela 3.2: Mapeamento de Propriedades DBLP para RPC

Propriedade DBLP	Propriedade RPC
dblp:authoredBy	rpc:author_inv
dblp:publishedBy	rpc:publisher
dblp:title	rpc:title / rpc:journal_name / rpc:book_title
dblp:yearOfPublication	rpc:year
rdfs:label	rpc:name
dblp:publishedBy	rpc:publisher

A parte do título foi tratada de forma especial, porque depende do tipo do Documento. A tabela a seguir resume o tratamento dado neste caso.

Tabela 3.3: Mapeamento da propriedade do Título

Tipo de Publicação RPC	Propriedade RPC Destino
Article_in_Journal	rpc:journal_name
Article_in_Proceedings	rpc:title
Book	rpc:book_title
Outros	rpc:title

No caso de um Documento não ter editora, é feita a associação ao *individual* `rpc:none`.

Capítulo 4

Fase 3 - Interrogações, Inferências e Página *Web*

O primeiro passo nesta última fase foi testar a ontologia Protégé. Abriu-se o `z01_meu_grafo_OUTPUT.rdf` e depois importou-se a ontologia feita na fase 1.

Ao inicializar o *Reasoner* não foi dada nenhuma inconsistência. Contudo, ao inferir sobre a classe `Unpublished`, não deu nenhum resultado, isto porque, na altura, o código da fase 2 estava a meter todas as publicações em `Publication` por padrão e não em `Document` (nem tratava o caso de não existir editora). Desta forma, como `Unpublished` é definido como sendo um documento que é publicado por uma editora `none`, a inferência não devolia nenhum resultado.

4.0.1 Inferências com DL

Depois de corrigir este problema, prosseguiu-se com as inferências na aba *DL Query*.

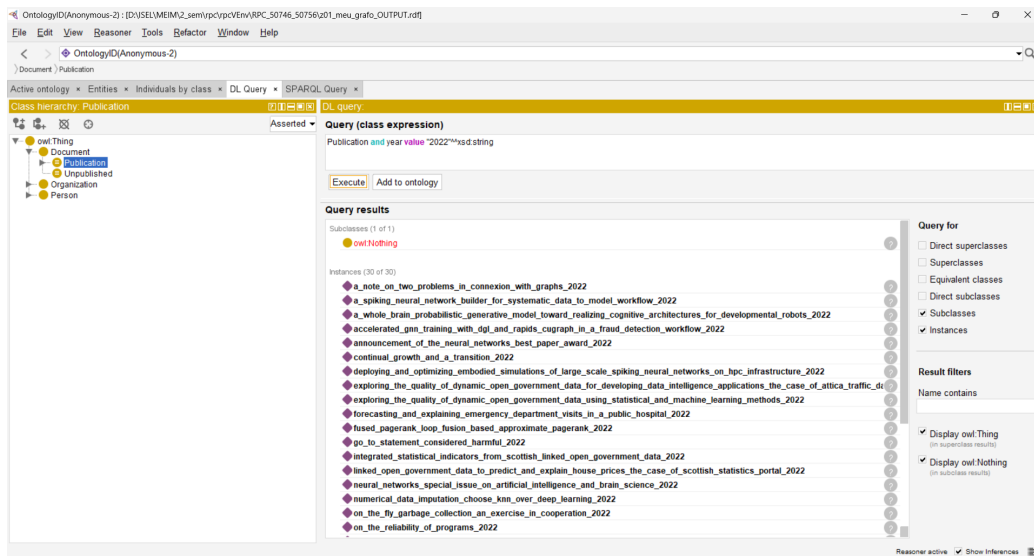


Figura 4.1: DL Query para ver obras de um ano específico

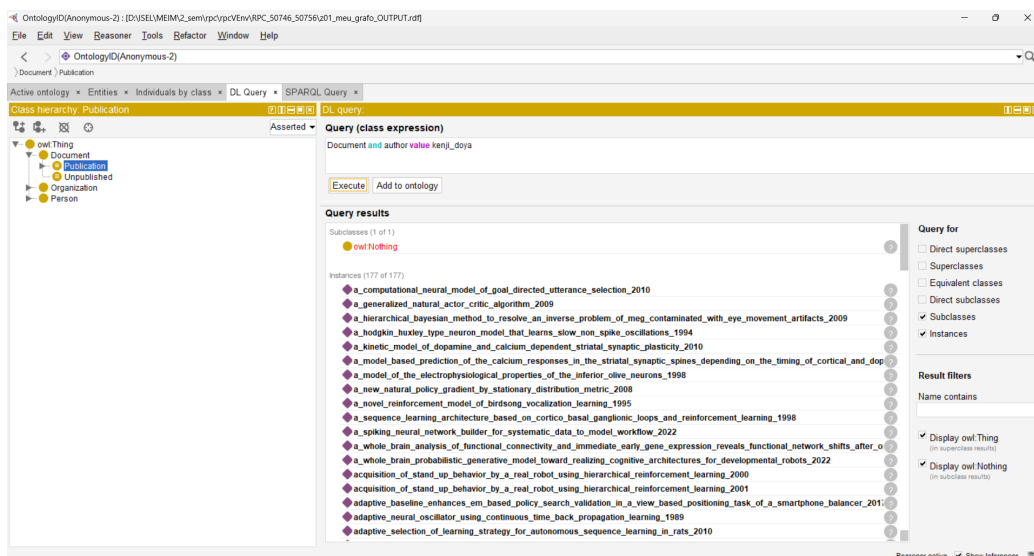


Figura 4.2: DL Query para ver obras de um autor específico

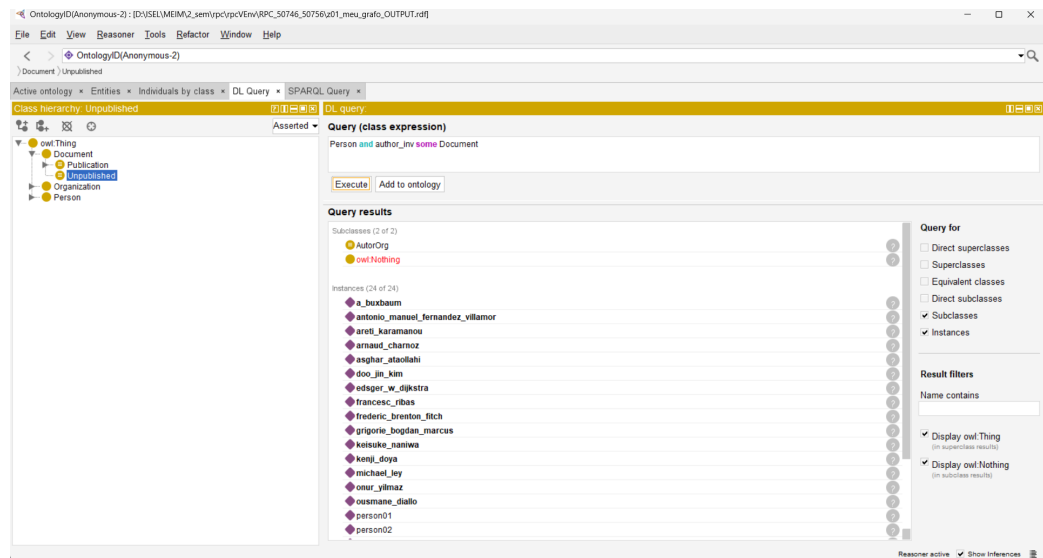


Figura 4.3: DL Query para ver autores existentes

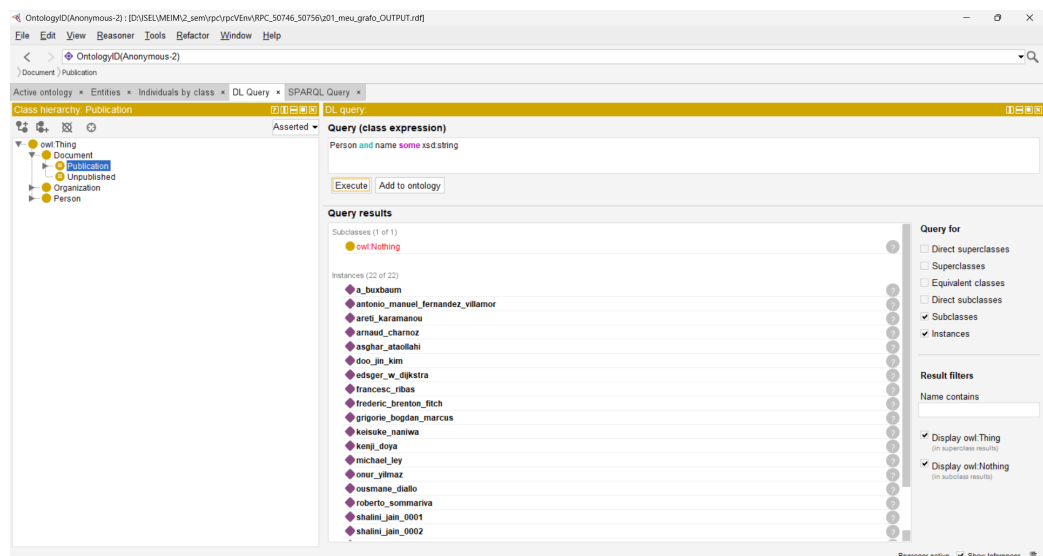


Figura 4.4: DL Query para ver que autores têm nomes

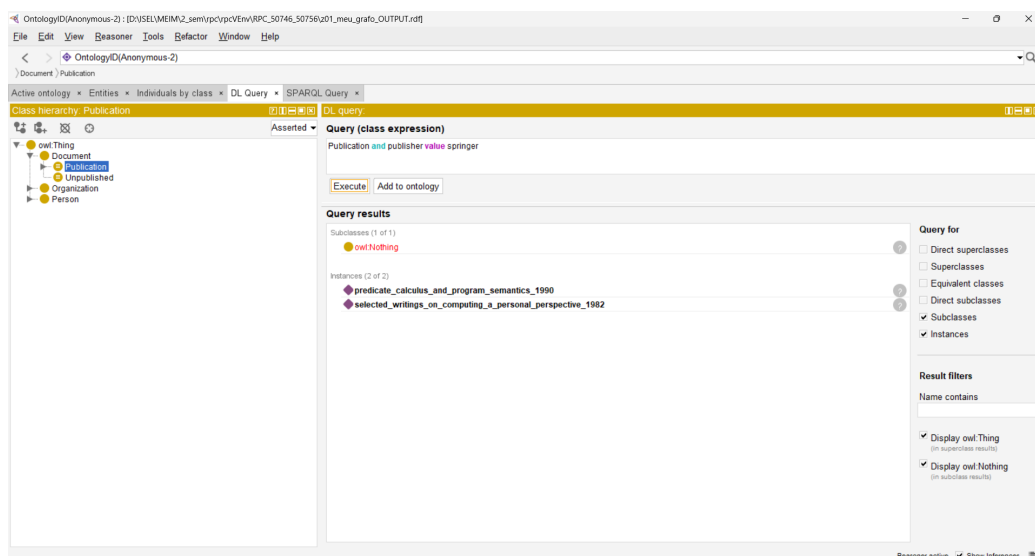


Figura 4.5: DL Query para ver obras de uma editora específica

Estas *DL Queries* estão disponíveis no ficheiro `z_DL_RPC.txt` na pasta do projeto.

4.0.2 Interrogações SPARQL

Em seguida, fez-se algumas interrogações SPARQL no RDF4J para aproveitar o facto de termos disponibilizado a ontologia num repositório lá.

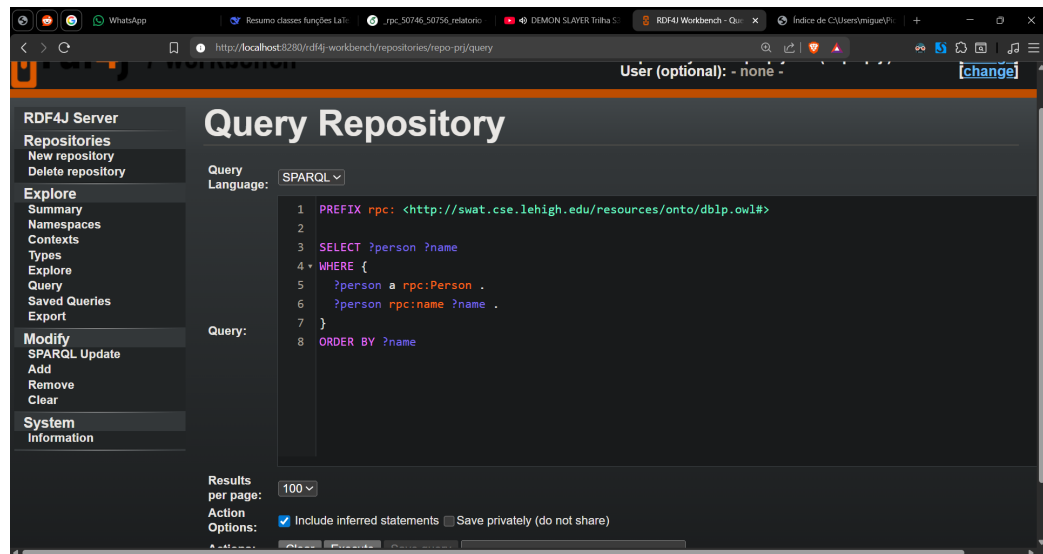


Figura 4.6: Query SPARQL para ver o nome de cada Person

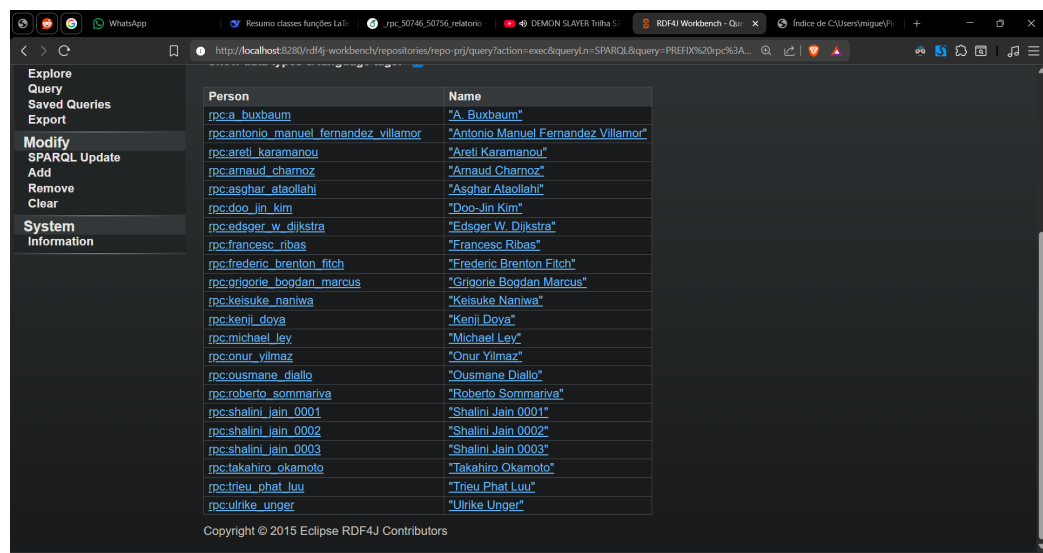


Figura 4.7: Resultado da Query SPARQL para ver o nome de cada Person

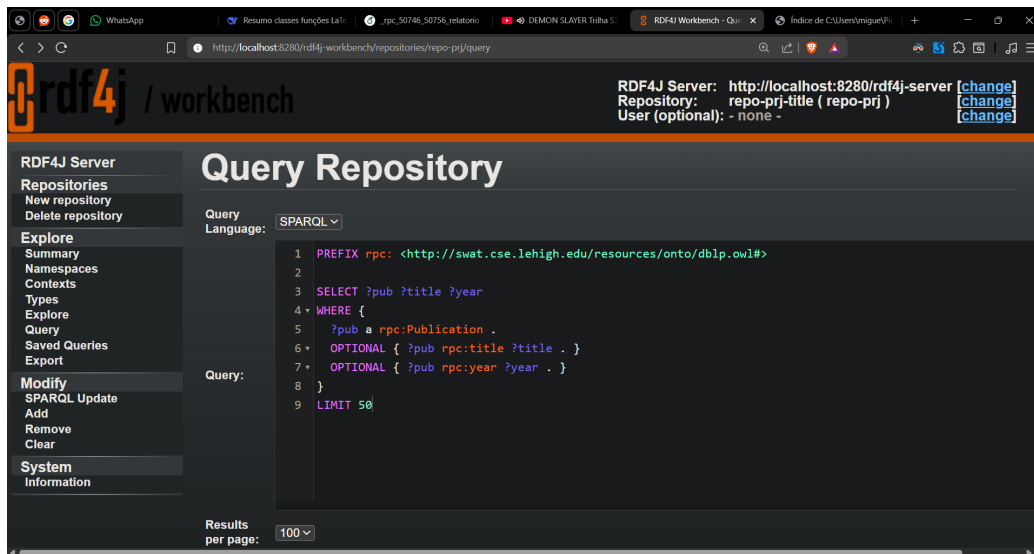


Figura 4.8: Query SPARQL para ver cada Publication, o seu título e ano da publicação

Pub	Title	Year
rpc:seisd_an_environment_for_extraction_of_semantic_information_from_on_line_dictionaries_1992	"SEISD: An environment for extraction of Semantic Information from on-line dictionaries."	"1992"
rpc:tge_tlinks_generation_environment_1994	"TGE: Tlinks Generation Environment."	"1994"
rpc:an_experiment_on_learning_appropriate_selectional_restrictions_from_a_parsed_corpus_1994	"An Experiment On Learning Appropriate Selectional Restrictions From A Parsed Corpus."	"1994"
rpc:acquisition_of_lexical_translation_relations_from_mrds_1994	"On Learning more Appropriate Selectional Restrictions."	"1994"
rpc:on_learning_more_appropriate_selectional_restrictions_1995	"On Learning more Appropriate Selectional Restrictions."	"1995"
rpc:communication_with_an_automatic_computer_1959	"Communication with an Automatic Computer"	"1959"
rpc:algol_sub_committee_report_extensions_1959	"Communication with an Automatic Computer"	"1959"
rpc:a_note_on_two_problems_in_connexion_with_graphs_1959	"Communication with an Automatic Computer"	"1959"
rpc:letter_to_the_editor_defense_of_algol_60_1961	"Letter to the editor: defense of ALGOL 60."	"1961"
rpc:some_meditations_on_advanced_programming_1962	"Some Meditations on Advanced Programming."	"1962"
rpc:operating_experience_with_algol_60_1962	"Some Meditations on Advanced Programming."	"1962"
rpc:some_comments_on_the_aims_of_mirfac_1964	"Some comments on the aims of MIRFAC."	"1964"
rpc:solution_of_a_problem_in_concurrent_programming_control_1965	"Solution of a problem in concurrent programming control."	"1965"
rpc:the_structure_of_the_the_multiprogramming_system_1967	"The structure of the 'THE' multiprogramming system."	"1967"
rpc:the_structure_of_the_multiprogramming_system_1968	"The structure of the 'THE' multiprogramming system."	"1968"
rpc:letters_to_the_editor_go_to_statement_considered_harmful_1968	"The structure of the 'THE' multiprogramming system."	"1968"

Figura 4.9: Resultado da Query SPARQL para ver cada Publication, o seu título e ano da publicação

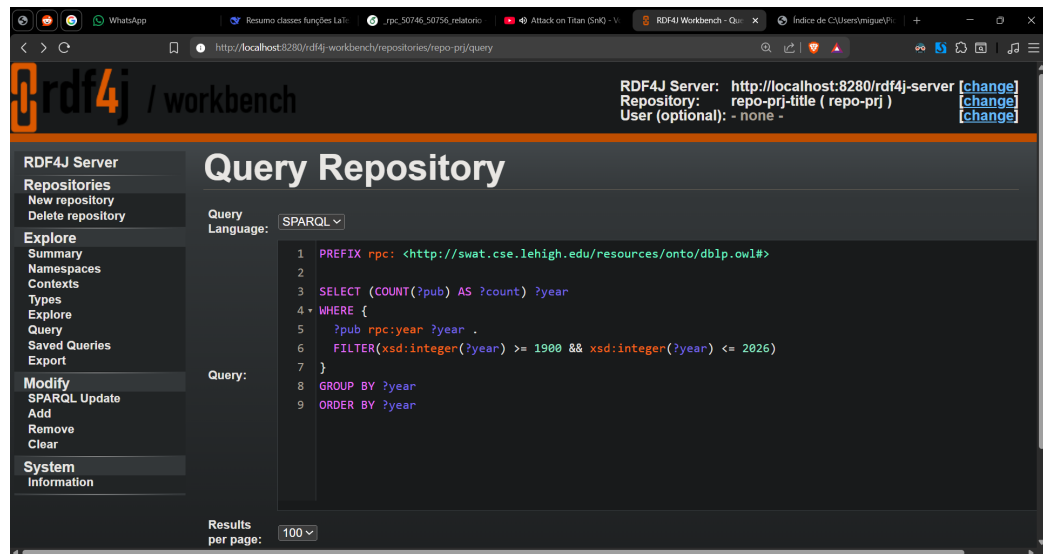


Figura 4.10: Query SPARQL para ver quantos livros se publicou em cada ano

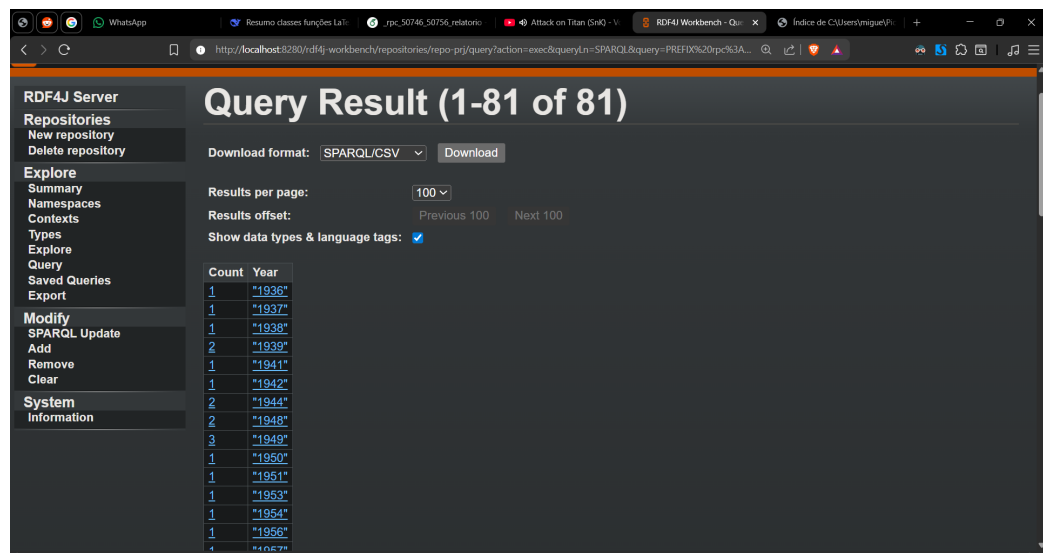


Figura 4.11: Resultado da Query SPARQL para ver quantos livros se publicou em cada ano

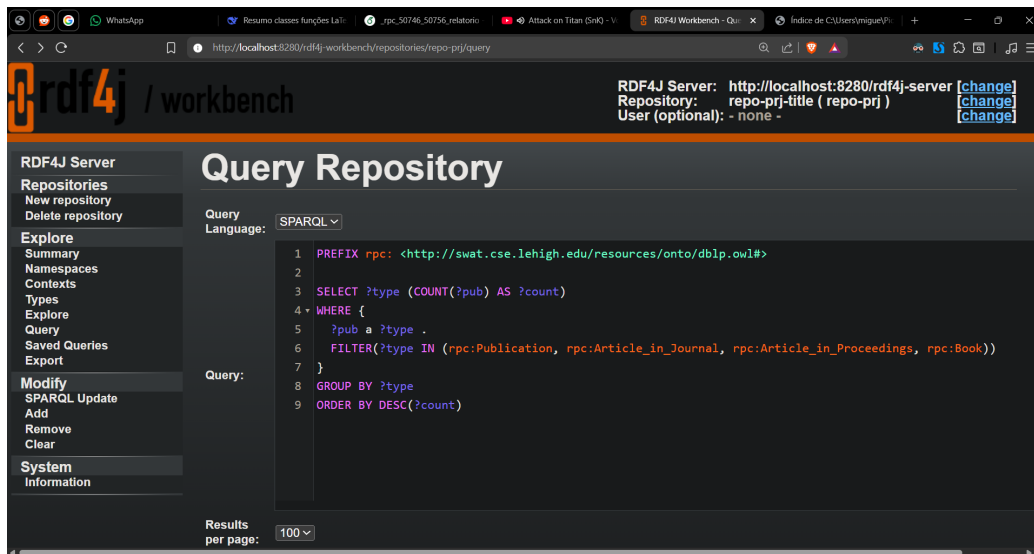


Figura 4.12: Query SPARQL para ver quantos documentos há de cada tipo

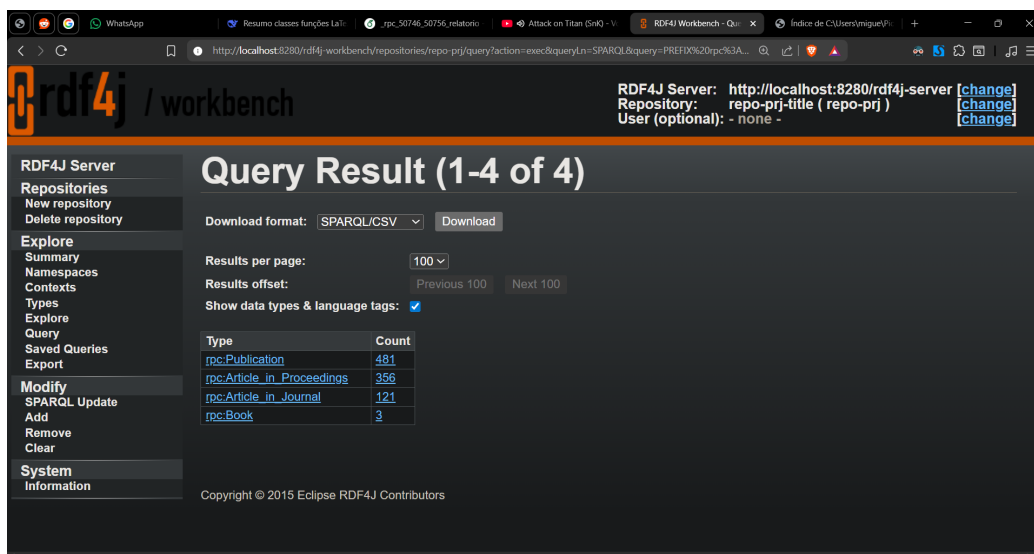


Figura 4.13: Resultado da Query SPARQL para ver quantos documentos há de cada tipo

Estas *SPARQL Queries* estão disponíveis no ficheiro `z_SPARQL_RPC.txt` na pasta do projeto.

4.0.3 Página *Web*

Esta última fase do trabalho foi feita tendo como apoio a Aula Prática 7. Os ajustes da página HTML foram feitos com auxílio de Inteligência Artificial para poupar tempo.

A partir do RDF4J, optou-se por usar a opção de "Exportar", que permite exportar toda a ontologia (TBox e ABox) num único ficheiro RDF (`export.rdf`), pois o exemplo da aula prática recorria a um ficheiro RDF e não a um *endpoint*.

A implementação desta etapa encontra-se no ficheiro `GG_fase3_cherry.py` que tem as seguintes funções disponíveis:

- `carregar_grafo_local()` - Carrega um grafo RDF a partir de um ficheiro local.
- `extrair_autores_publicacoes()` - Extrai todos os autores e os títulos das suas publicações. Retorna um dicionário onde a chave é o nome do autor e o valor contém o nome, URI e lista de publicações.
- `extrair_informacao_detalhada()` - Extrai informação detalhada sobre publicações (título, ano, tipo, publishers, journal). Se fornecida uma URI de autor, filtra apenas as publicações desse autor.
- `class Main()` - Classe responsável pela aplicação *web* com recurso ao CherryPy.

Métodos expostos (rotas HTTP):

- `index()` — Página principal. Mostra a lista de todos os autores com *links* para as suas publicações.
- `publicacoes(self, nome=None)` — Página que mostra as publicações de um autor específico.
- `todas_publicacoes(self)` — Página que lista todas as publicações presentes no grafo, sem filtro por autor.

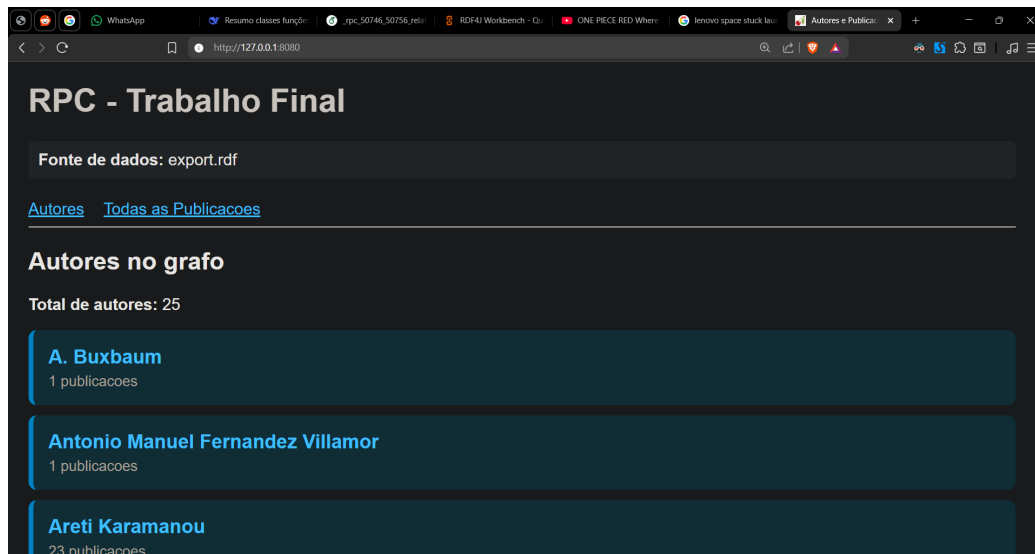


Figura 4.14: Página principal da aplicação *web*.

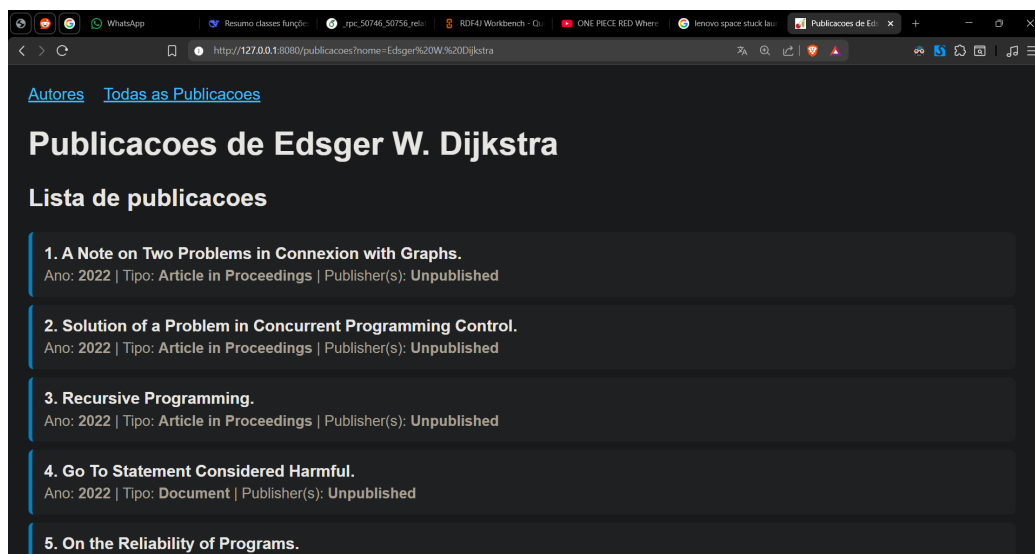


Figura 4.15: Página com as publicações de Edsger W. Dijkstra.

Capítulo 5

Conclusão

Este trabalho permitiu desenvolver um repositório de referências bibliográficas baseado numa ontologia própria, adaptada a partir do DBLP e da ontologia SWAT. Na primeira fase, construiu-se uma TBox com os conceitos fundamentais, propriedades e restrições, sendo que a utilização do Protégé serviu para validar e testar as definições.

Na segunda fase, a criação da ABox recorrendo a dados do DBLP através de *queries* SPARQL. Apesar das limitações relacionadas à indisponibilidade de alguns *endpoints* e à necessidade da realização de mapeamentos, foi possível construir um grafo local com a ontologia definida.

Por fim, na terceira fase, a disponibilização da ontologia como um SPARQL-*endpoint* no RDF4J e o desenvolvimento de uma aplicação *web* com CherryPy. permitiram realizar inferências e interrogações, em DL e em SPARQL, respetivamente, e apresentar os resultados de uma forma mais apelativa.